

AI ASSISTANT APP

PROBLEM STATEMENT:

In today's fast-paced digital world, users require quick, efficient, and intelligent assistance for handling daily tasks, answering queries, and improving productivity. Traditional mobile applications are often limited to predefined functionalities and lack the ability to understand and respond to user queries dynamically.

With the growing demand for smart systems, there is a need for an AI-powered mobile assistant that can interact with users in a natural and conversational manner. Existing solutions may be complex, not user-friendly, or lack seamless integration within a lightweight mobile application.

The problem is to design and develop a user-friendly AI Assistant mobile application that:

- Provides real-time conversational interaction
- Integrates with an AI API for intelligent responses
- Maintains smooth performance with efficient state management
- Offers an intuitive and visually appealing chat interface

The system should be capable of understanding user inputs, processing them through an external AI service, and delivering meaningful responses in a structured and responsive manner.

ABSTRACT

The rapid advancement of artificial intelligence has significantly transformed the way users interact with mobile applications. This project presents the design and development of an AI-powered assistant mobile application built using Flutter, aimed at providing real-time, intelligent, and conversational user support.

The application enables users to communicate through a chat-based interface, where user inputs are processed and sent to an external AI API for generating meaningful responses. The system integrates efficient state management using the Provider framework to ensure smooth and responsive user interactions. The architecture of the application is modular, consisting of presentation, state management, service, and data layers, which enhances scalability and maintainability.

The app features a clean and user-friendly interface, including authentication screens, a chat interface with message bubbles, and seamless navigation across different screens. By leveraging API integration and asynchronous data handling, the system delivers real-time responses while maintaining optimal performance.

This project demonstrates how modern mobile technologies and AI services can be combined to build an intelligent assistant capable of enhancing user experience, improving accessibility to information, and supporting future extensions such as voice interaction and personalized recommendations.

TECHNOLOGIES USED

The AI Assistant mobile application is developed using modern technologies and frameworks to ensure scalability, performance, and a smooth user experience. The key technologies used in this project are as follows:

1. Flutter

Flutter is an open-source UI toolkit developed by Google for building natively compiled applications from a single codebase.

- Used for developing the entire mobile application
- Provides a rich set of pre-designed UI components
- Ensures cross-platform compatibility (Android & iOS)

2. Dart

Dart is the programming language used by Flutter for building fast and efficient applications.

- Used for writing application logic
- Supports asynchronous programming
- Helps in building responsive and dynamic UI

3. Provider (State Management)

Provider is a lightweight state management solution used in Flutter.

- Manages application state efficiently
- Updates UI dynamically when data changes
- Simplifies data flow between components

4. REST API Integration

The application communicates with an external AI service using REST APIs.

- Sends user input to the AI server
- Receives and processes responses in JSON format
- Enables real-time conversational functionality

5. HTTP Package

Flutter's HTTP package is used to handle network requests.

- Performs POST requests to the API
- Handles API responses and error management
- Supports asynchronous communication

6. Firebase (Authentication & Backend Services)

Firebase, developed by Google, is used for backend support.

- User authentication (Login/Register)
- Secure user session management
- Scalable cloud-based backend

7. Android Studio / VS Code

Development tools used for building and testing the application.

- Code editing and debugging
- Emulator support for testing
- Plugin support for Flutter development

8. Git & GitHub

Version control and code hosting platform.

- Tracks code changes
- Enables collaboration
- Stores project repository online

METHODOLOGY

The development of the AI Assistant mobile application follows a systematic and modular approach to ensure efficiency, scalability, and ease of maintenance. The methodology adopted combines modern mobile development practices with API-based AI integration.

1. Requirement Analysis

In this phase, the system requirements were identified and analyzed.

- Need for a conversational AI assistant
- Real-time response handling
- User-friendly chat interface
- Secure user authentication
- Efficient state management

This step ensured a clear understanding of the problem and defined the project scope.

2. System Design

The system was designed using a modular architecture to separate concerns and improve maintainability.

- UI Layer (Screens & Widgets)
- State Management Layer (Provider)
- Service Layer (API handling)
- Data Layer (Models)

The design ensures smooth communication between components and scalability for future enhancements.

3. UI/UX Development

The user interface was developed using Flutter to provide a clean and interactive experience.

- Chat-based interface with message bubbles
- Input field for user queries

- Smooth navigation between screens (Login, Home, Chat)
- Responsive layout for different devices

The focus was on simplicity, readability, and user engagement.

4. API Integration

The application integrates with an external AI service using REST APIs.

- User messages are sent to the API via HTTP requests
- The API processes the input and generates a response
- Responses are received in JSON format and displayed to the user

This enables real-time conversational functionality.

5. State Management Implementation

Provider was used to manage application state efficiently.

- Stores chat messages
- Updates UI dynamically when new messages arrive
- Handles loading and response states

This ensures smooth performance and organized data flow.

6. Authentication System

Firebase Authentication was implemented to manage user access.

- User registration and login functionality
- Secure session handling
- Persistent authentication state

This ensures that only authorized users can access the application.

7. Testing and Debugging

The application was tested to ensure functionality and performance.

- Debugging using Flutter tools
- Testing API responses and error handling
- Fixing build and device connection issues

This phase ensured reliability and stability of the application.

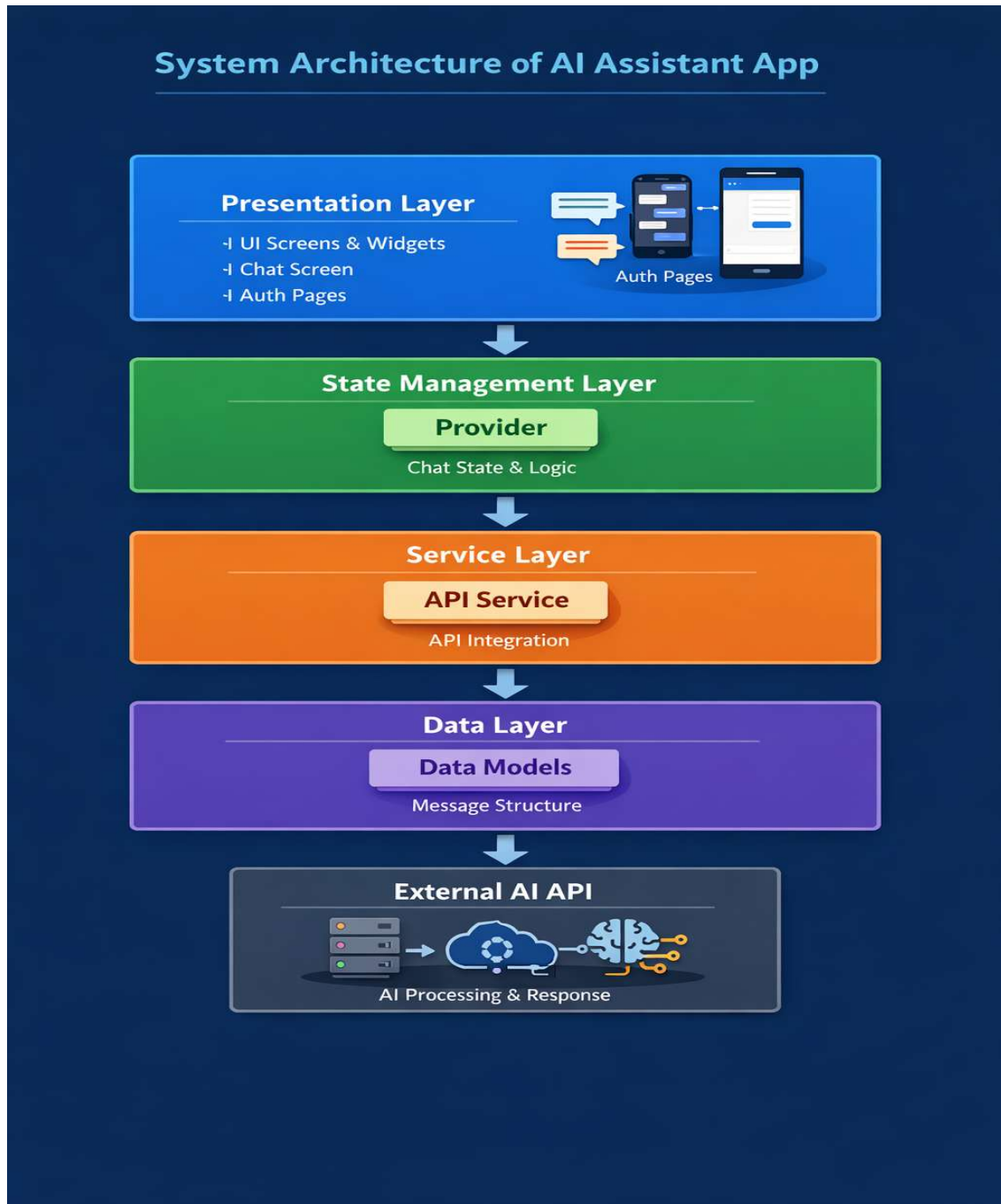
8. Deployment

The final application was built and deployed on an Android device.

- APK generation using Flutter
- Testing on real device
- Preparing project for GitHub submission

SYSTEM ARCHITECTURE

The system follows a layered architecture where UI interacts with Provider for state management, which communicates with the API service to fetch AI responses and update the interface dynamically.

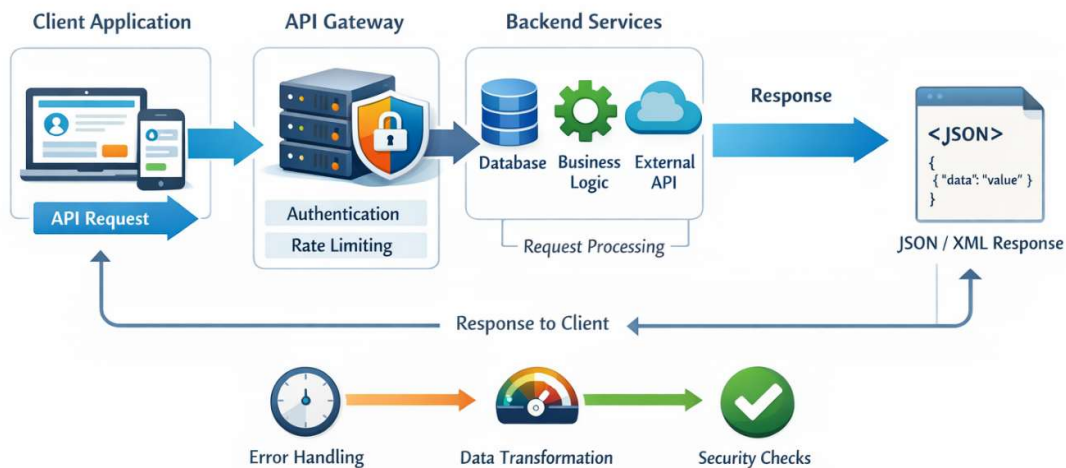


APP FLOW

The API flow describes how user messages are processed and how responses are generated in the AI Assistant application.

1. The user enters a message in the chat interface.
2. The message is sent to the Chat Provider, which manages the app state.
3. The provider forwards the message to the API Service.
4. The API Service sends an HTTP request to the external AI API.
5. The AI API processes the request and generates a response.
6. The response is returned in JSON format to the API Service.
7. The provider updates the message list with the AI response.
8. The updated response is displayed in the chat UI.

API Flow



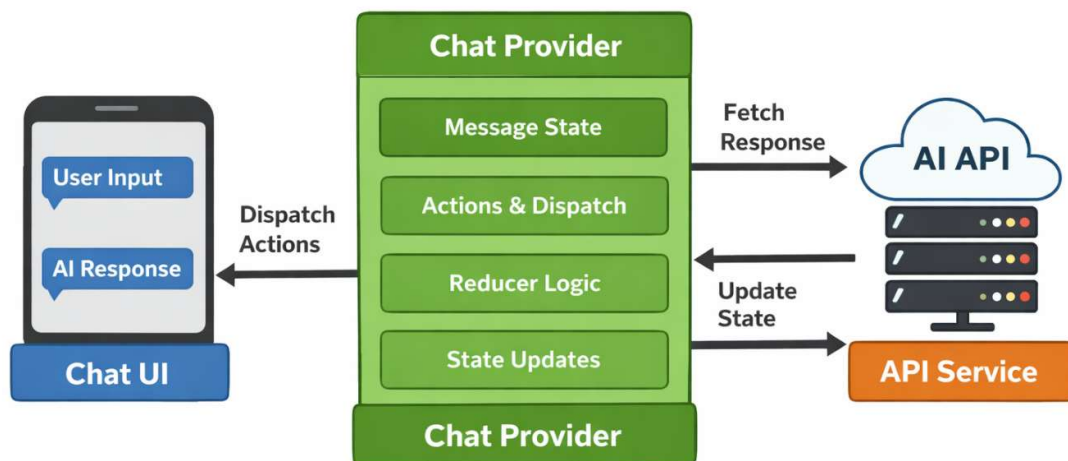
STATE MANAGEMENT

State management in the AI Assistant app is handled using the Provider pattern, which acts as a central controller between the user interface and the API service.

When a user enters a message in the chat UI, the request is sent to the Chat Provider, which manages the message state and application logic. The provider then forwards the request to the API Service to fetch a response from the AI API. Once the response is received, the provider updates the state and notifies the UI to display the new message.

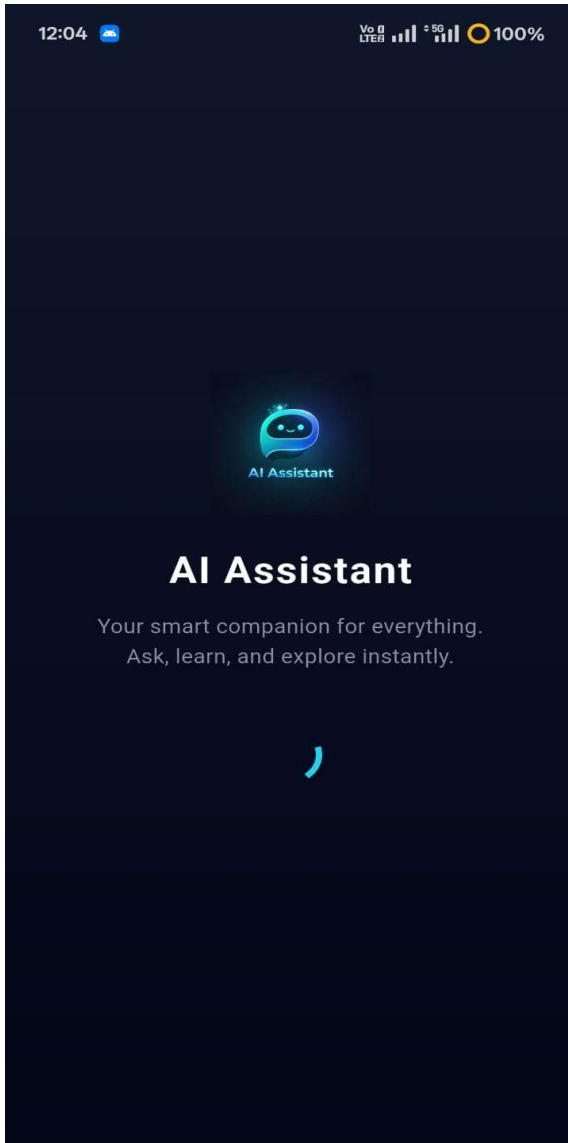
This approach ensures a smooth data flow, efficient UI updates, and better separation of concerns, making the application scalable and easy to maintain.

State Management in AI Assistant App

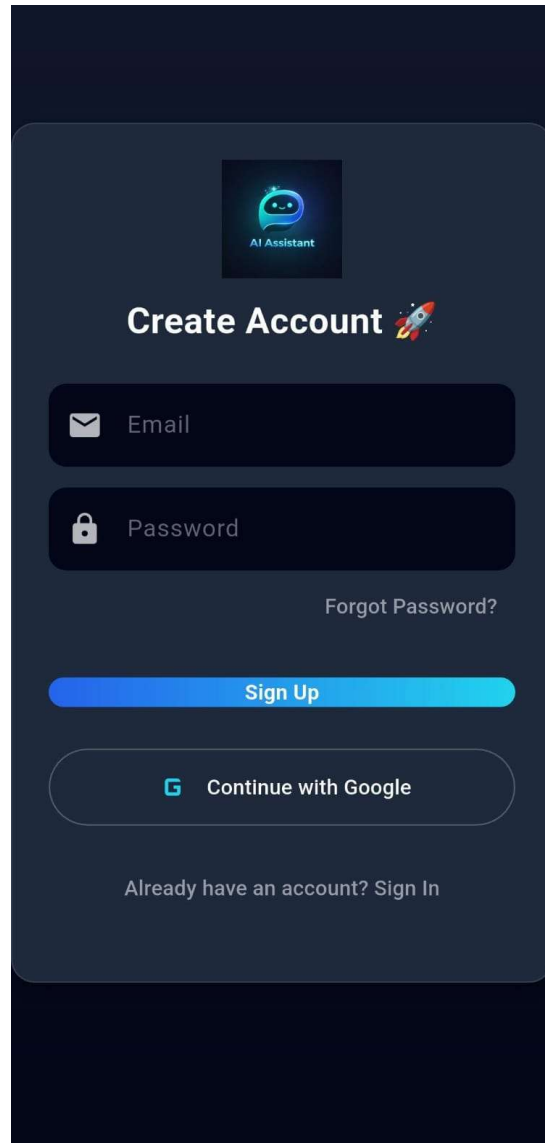


UI SCREENS

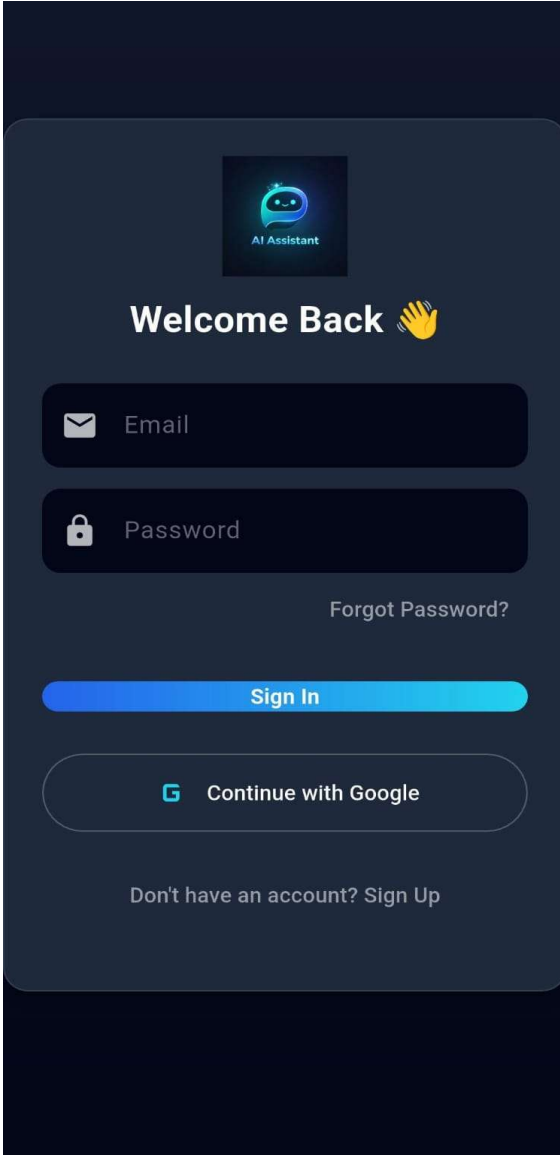
Splash Screen



New Account Creation

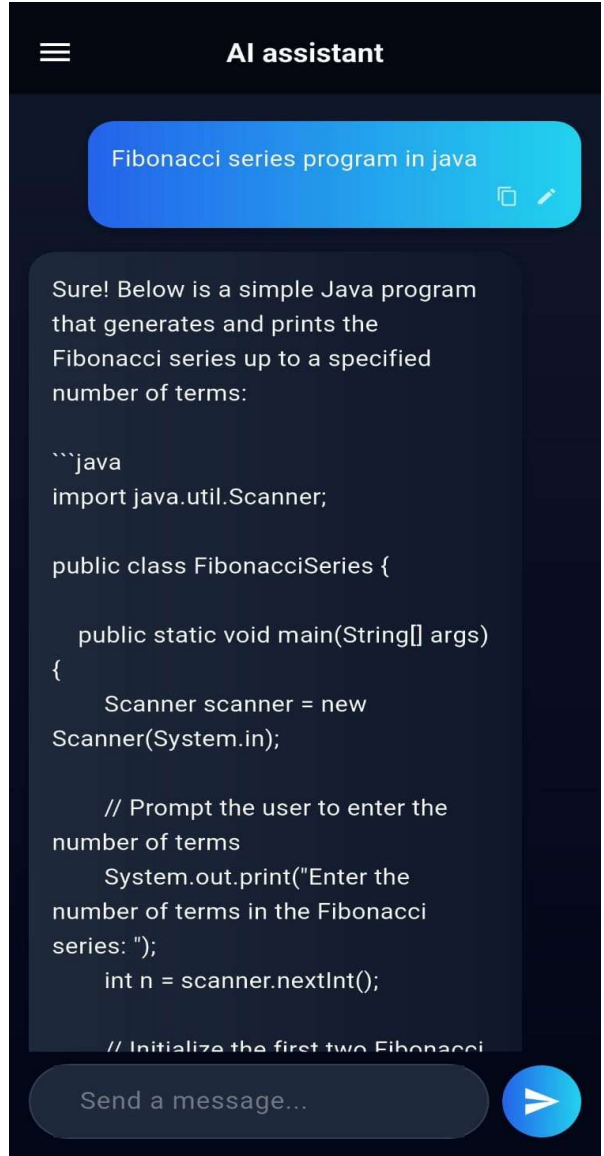


Sign In screen



The Sign In screen features a dark blue background. At the top center is a logo for 'AI Assistant' with a stylized face. Below the logo, the text 'Welcome Back' is displayed in white, followed by a yellow hand icon. There are two input fields: 'Email' with an envelope icon and 'Password' with a lock icon. A link 'Forgot Password?' is positioned to the right of the password field. A large blue button labeled 'Sign In' is centered below the input fields. Below the button is a 'Continue with Google' option, represented by the Google 'G' logo and text. At the bottom, there is a link 'Don't have an account? Sign Up'.

Chat with AI



The Chat with AI screen has a dark blue background. At the top, there is a hamburger menu icon and the text 'AI assistant'. A blue message bubble contains the text 'Fibonacci series program in java'. Below this, a response from the AI assistant is shown in a light blue bubble, stating: 'Sure! Below is a simple Java program that generates and prints the Fibonacci series up to a specified number of terms:'. The response includes a code block for a Java program that generates the Fibonacci series. At the bottom, there is a text input field labeled 'Send a message...' and a blue circular button with a white right-pointing arrow.

```
``java
import java.util.Scanner;

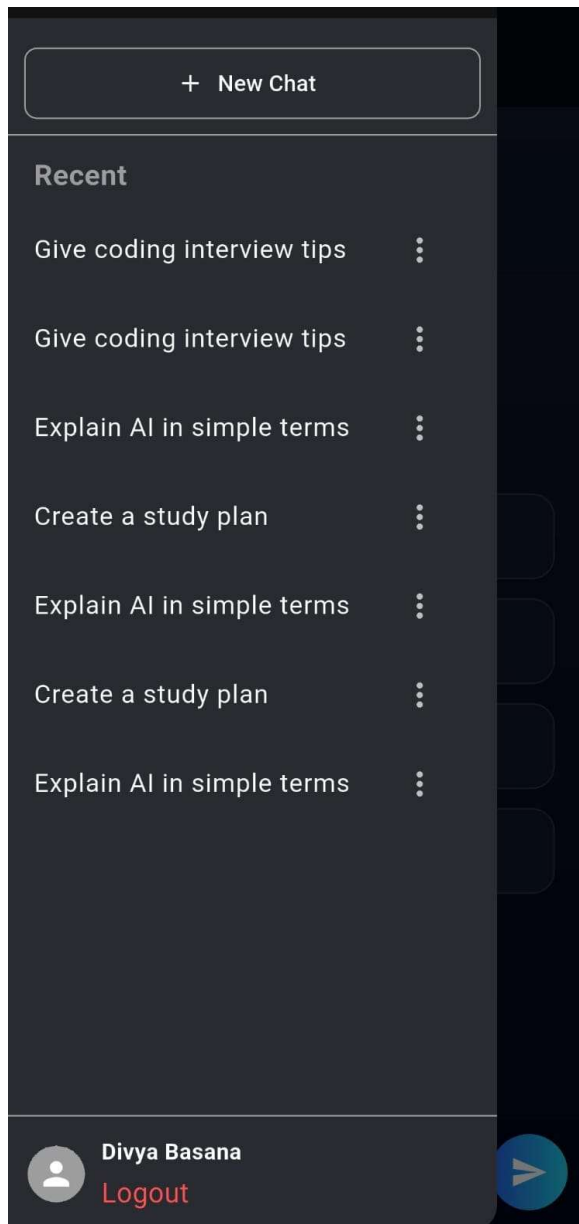
public class FibonacciSeries {

    public static void main(String[] args)
    {
        Scanner scanner = new
        Scanner(System.in);

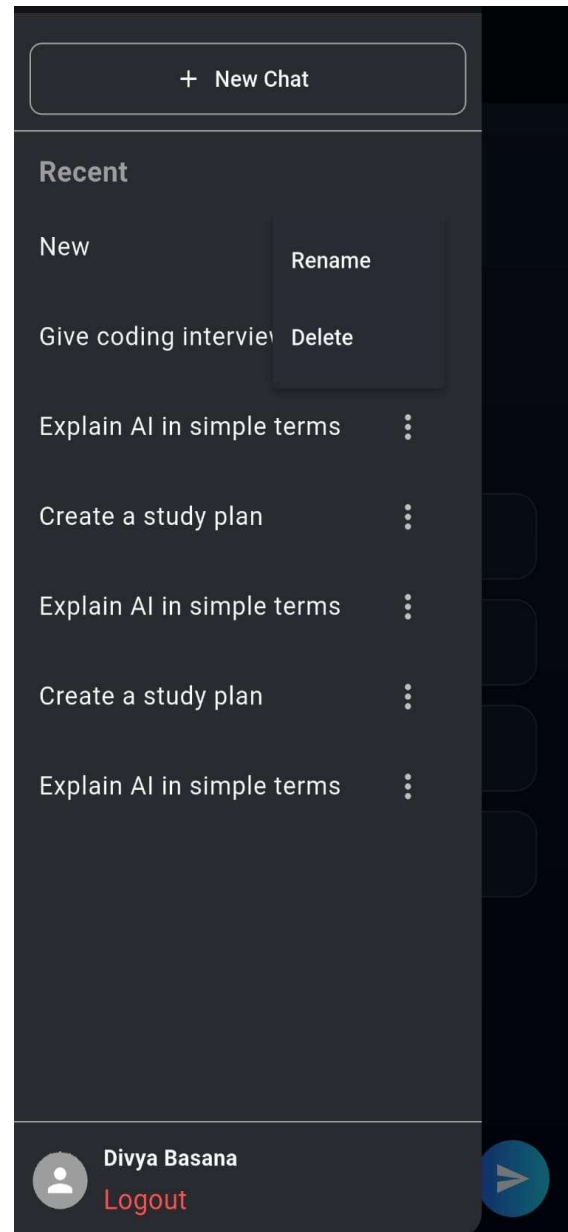
        // Prompt the user to enter the
        number of terms
        System.out.print("Enter the
        number of terms in the Fibonacci
        series: ");
        int n = scanner.nextInt();

        // Initialize the first two Fibonacci
```

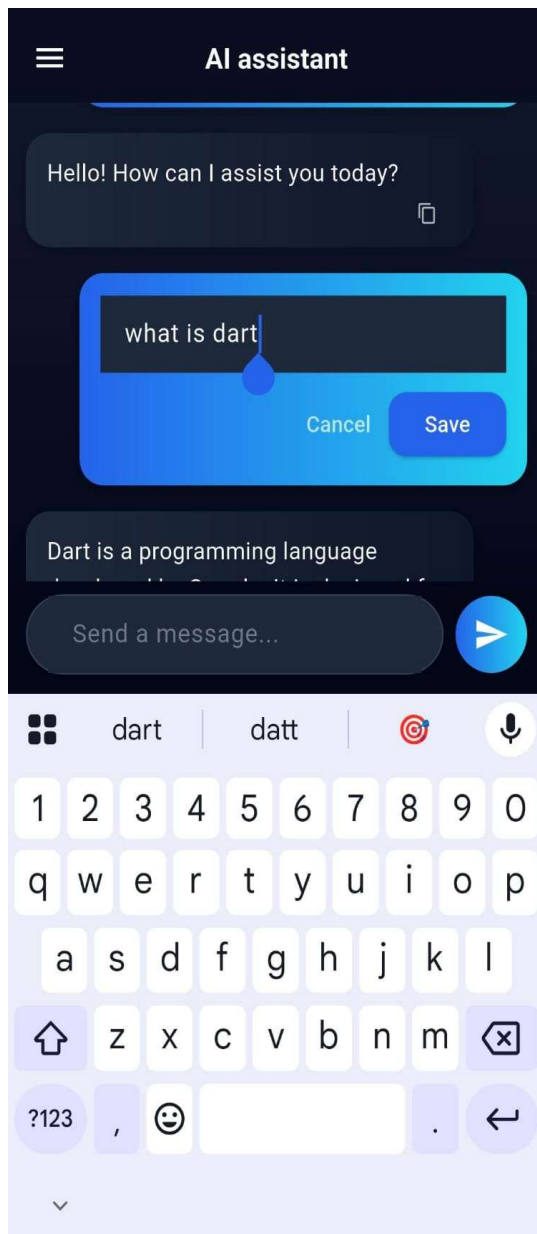
Chat History



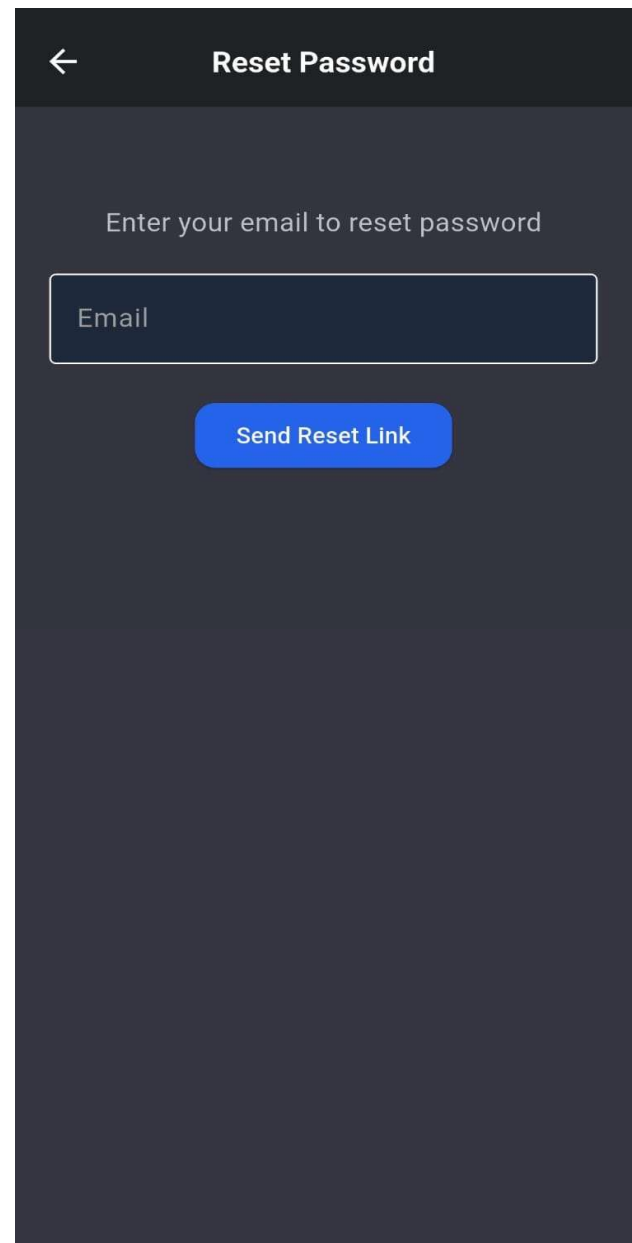
Rename & Delete Chat



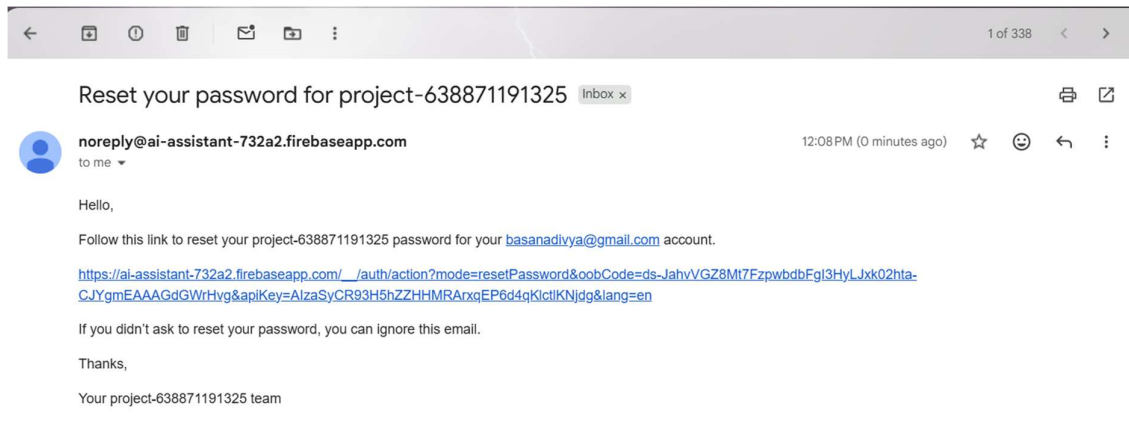
Edit the user prompt



Forget Password



Forget Password Rest Link to mail



Reset Password

Reset your password

for **basanadivya@gmail.com**

New password 👁

Password changed

Password changed

You can now sign in with your new password

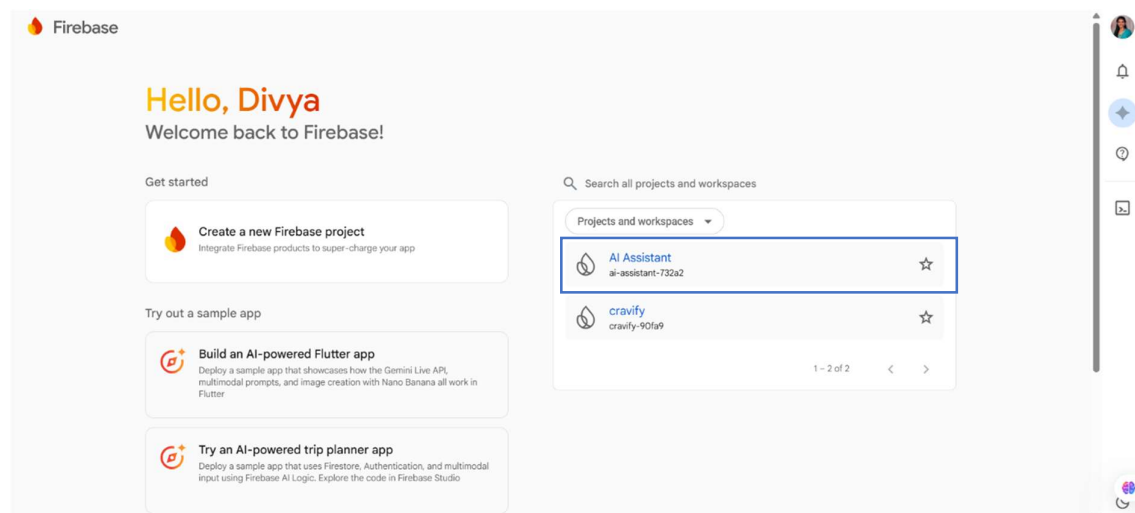
Firestore - Backend

Firestore is a cloud-based backend platform developed by Google that provides tools and services for building scalable and secure applications.

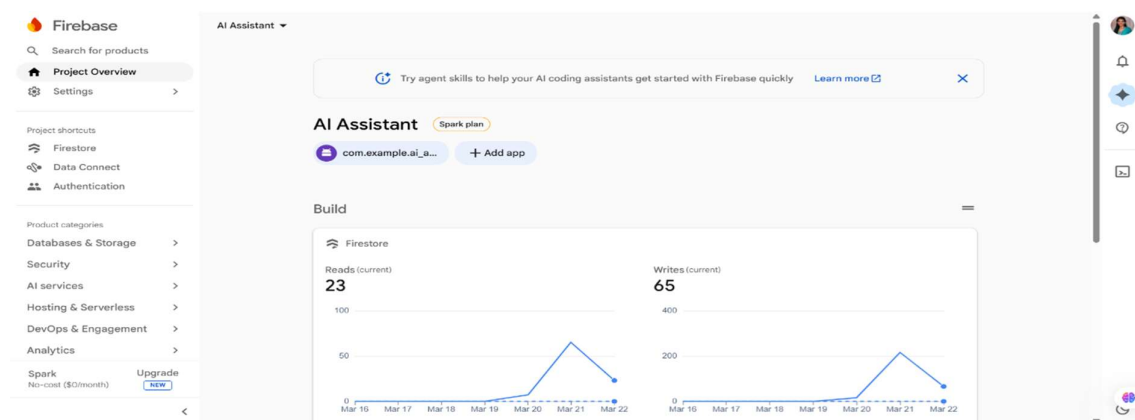
In this project, Firestore is used to handle user authentication and backend support. It enables users to register and log in securely using email and password. Firestore manages user sessions, ensuring that authentication state is maintained even after restarting the application.

The integration of Firestore simplifies backend development by eliminating the need for a custom server and provides reliable, real-time services. It also ensures data security, scalability, and seamless user management.

Firestore Console



Project Overview



Firebase Authentication

Search for products

Project Overview

Settings

Project shortcuts

Authentication

Data Connect

Firestore

Product categories

Databases & Storage

Security

AI services

Hosting & Serverless

DevOps & Engagement

Analytics

AI Assistant

Authentication

Users | Sign-in method | Templates | Usage | Settings | Extensions

The following Authentication features will stop working when Firebase Dynamic Links shuts down soon: email link authentication for mobile apps, as well as Cordova OAuth support for web apps.

Search by email address, phone number, or user UID

Add user

Identifier	Providers	Created	Signed in	User UID
divyawork2525@gmail...		Mar 22, 2026	Mar 22, 2026	4LVPv7iVoKTPONKqgFYjEv...
23335a0702@mvgrce...		Mar 22, 2026	Mar 23, 2026	qwq14H3zOhPBLfttoHn4jU...
ab@gmail.com		Mar 22, 2026	Mar 22, 2026	4LsHOUNjMNOHDCCcXIXYr...
hjj@gmail.com		Mar 22, 2026	Mar 22, 2026	d9rfqMLTSpNsCzo5i4yvdP0...
basanadivya@gmail.com		Mar 22, 2026	Mar 23, 2026	AVwqTcSWc5bJvNTPvPzXhOP...

Firestore Data

Search for products

Project Overview

Settings

Project shortcuts

Firestore

Data Connect

Authentication

Product categories

Databases & Storage

Security

AI services

Hosting & Serverless

DevOps & Engagement

Analytics

AI Assistant

Cloud Firestore > Database (default)

users > 6eecMWzCXD...

More in Google Cloud

(default)

+ Start collection

users

+ Add document

6eecMWzCXDW1wRAharcFBpEgwiQ2

8IehqMRfqPbHuavqeHtPCFVSdGq2

Bkb51VDIAyUzJfxinse8EqGZ3w1

EUJ5sYbKxNVDz7EoaaJmDySF02

hNSGcWBwhxS3LhdujPN3TrVDZDJ2

kTrvssnH7RQdtypbobPH6FFgGYt2

oT8LPL40GH0uSWDAQMVq7uFrLVx2

ymb6u8yAC4eeWQhc0Ln5W8eVL6G2

6eecMWzCXDW1wRAharcFBpEgwiQ2

+ Start collection

chats

+ Add field

1

id: "1774068771832"

messages

0

isUser: true

text: "about flutter"

1

isUser: false

text: ""

title: "About flutter"

CONCLUSION

The AI Assistant mobile application successfully demonstrates the integration of modern mobile development technologies with artificial intelligence to provide an interactive and intelligent user experience. The application enables users to communicate through a chat-based interface, where queries are processed in real time using an external AI API.

The system is built using Flutter and follows a modular architecture that includes presentation, state management, service, and data layers. This structured approach ensures maintainability, scalability, and efficient performance. The use of Provider for state management allows smooth data flow and dynamic UI updates, while Firebase integration ensures secure user authentication and session handling.

Overall, the project achieves its objective of developing a user-friendly AI assistant capable of delivering meaningful responses and enhancing user interaction. It also provides a strong foundation for future enhancements such as voice input, personalization, and advanced AI capabilities.